Transactions

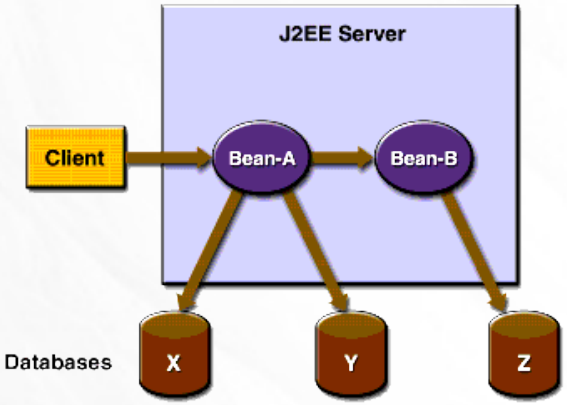
Chapter 17

Transactions and EJBs

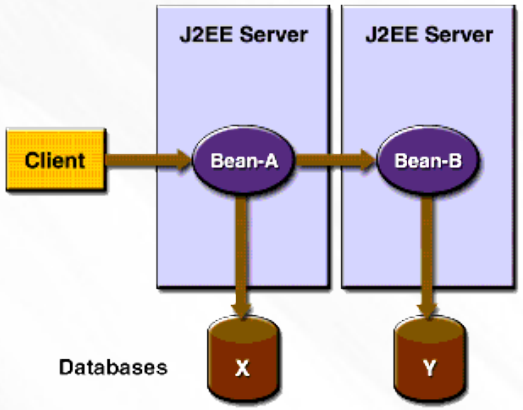
- Transactions are intended to increase application reliability
- A group of activities either all succeed or are all rolled back
  - debit checking account, credit savings account
- Desirable properties from database theory:
  - atomicity, consistency, isolation, durability (ACID)

Transactions With Multiple Databases

- Java EE handles multiple database transactions automatically with container-managed transactions



- Even across multiple J2EE servers



TravelAgent Example

- bookPassage(CreditCardDO c, double price) has following tasks
  - check inputs for existence
    - customer, cruise, cabin
  - create reservation
  - process payment
  - create ticket and return to customer
- What could go wrong?

ACID Transactions

- Atomic
  - execute completely or not at all
- Consistent
  - data after operation must be in harmony with real-world represented by system
- Isolated
  - data accessed by transaction not affected by other parts of the system
- Durable
  - changes to data not lost if system crashes

Is TravelAgent EJB Atomic?

- Does it ensure it executes completely or not at all?
- Critical things are those actions that change or create information
  - create reservation, process payment, create ticket
- If process payment or create ticket fails, need to roll back reservation creation

Is TravelAgent EJB Consistent?

- This is not only getting the transaction attrubutes correct
  - eliminating bugs in the application
  - enforcing referential integrity constraints in database
- Should not be possible to create reservation without valid cabin, cruse and customer foreign keys

Is TravelAgent EJB Isolated?

- To be isolated transaction must be protected from other processes accessing the data being modified
- EJB transactions will isolate the bean instance and database row being modified

Is TravelAgent EJB Durable?

- To be durable, all changes and new data must be persisted
  - not just modified in memory
- Transaction is not finished until this happens
- EJB transactions are designed to support this
  - big convenience over doing by hand

Declarative Transaction Management

- EJB specification allow transactions to be defined declaratively
  - with attributes or deployment descriptor
    - @jakarta.ejb.TransactionAttribute
  - easier (less error prone) than using programmatic transactions with an API
  - Java has Java Transaction Service (JTS) API for controlling transactions in the code
- Can set the transaction attributes for a bean or specific methods in a bean

Container Managed Transactions

- Applicable to all types of EJBs
- Transaction starts just before an EJB method is called
  - commits just before method ends
- Nested or multiple transactions in one method not allowed
- You control which bean methods associated with transactions

Transaction Attributes Specified in Deployment Descriptor

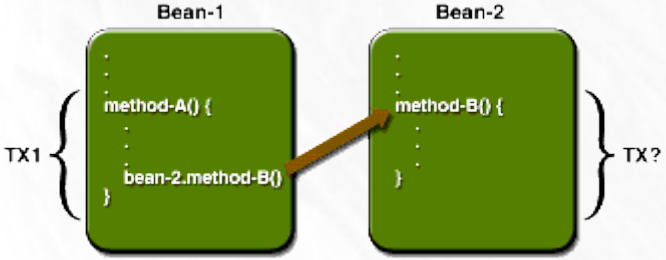
- Session beans need definitions for business methods but not for create methods
- Entity beans need definitions for business, create, remove and finder methods
- Message-driven beans need definitions for onMessage method
  - either Required or NotSupported

Transaction Scope

- Transaction scope means the EJBs that participate in a transaction
- In bookPassage() method of TravelAgent, all of the effected EJBs should be in the same transaction scope
- Transaction is propagated to EJB when its method is called and included in the scope of the calling EJB
- Transaction can end if exception is thrown as well as normal method completion

Transaction Attributes and Scope

- Suppose method-A calls method-B in a different bean



- Transaction attribute defines how method-B participates in method-A transaction
  - NotSupported, Supports, Required, RequiresNew, Mandatory, Never
  - declared in @TransactionAttribute or ejb-jar.xml

Setting Transaction Attributes

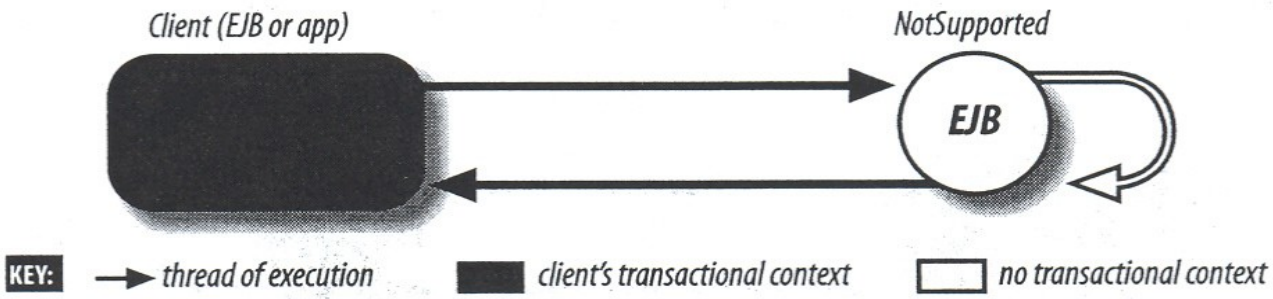
- `public enum TransactionAttributeType {MANDATORY, REQUIRED, REQUIRES_NEW, SUPPORTS, NOT_SUPPORTED, NEVER}`
- `@TransactionAttribute`
  - `TransactionAttributeType value() default TransactionAttributeType.REQUIRED`
- Applied per method or on bean class

Setting Transaction Attributes

- Transaction attributes can be set in <assembly-descriptor> section of ejb-jar.xml file
  - in list of <container-transaction> elements
- Each <container-transaction> element has
  - <method> element with <ejb-name> and <method-name> (can be *)
  - <trans-attribute> element with the transaction attribute that applies

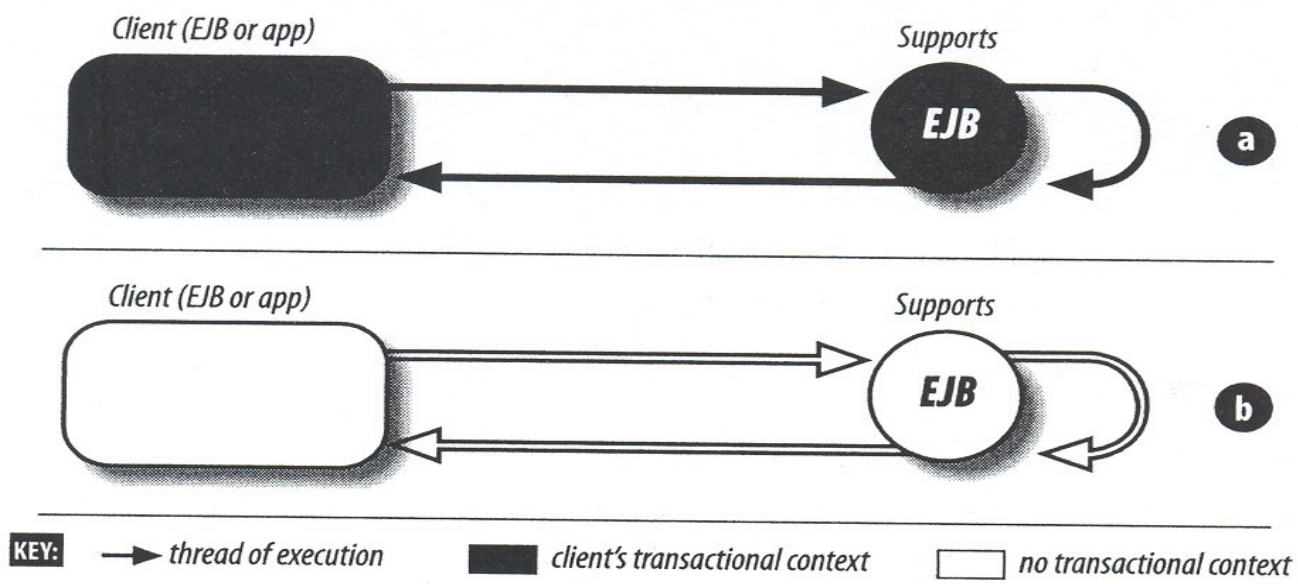
Transaction Attributes: NotSupported

- If client running within transaction and calls method
  - container suspends client's transaction
  - runs method
  - after method completes, container resumes client's transaction
- Client not associated with transaction
  - method does not run in transaction
- Use for methods not requiring transaction



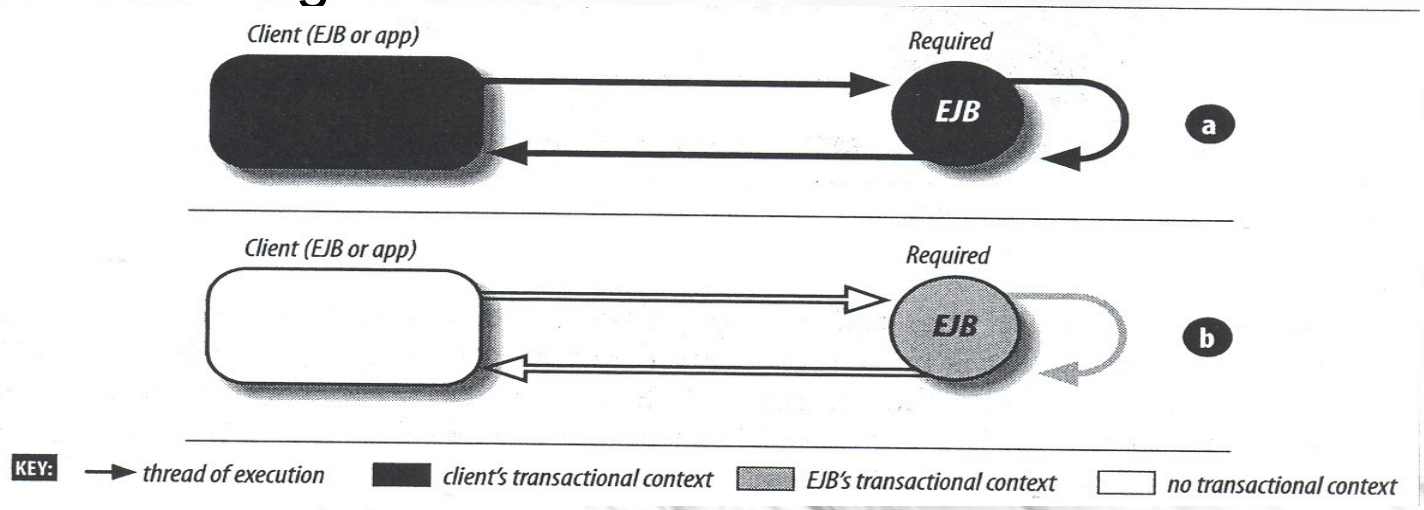
Transaction Attributes: Supports

- Client running within transaction
  - method runs in client's transaction
- Client not associated with transaction
  - method does not run in transaction
- Use with caution because transactional behaviour of method depends on caller



Transaction Attributes: Required

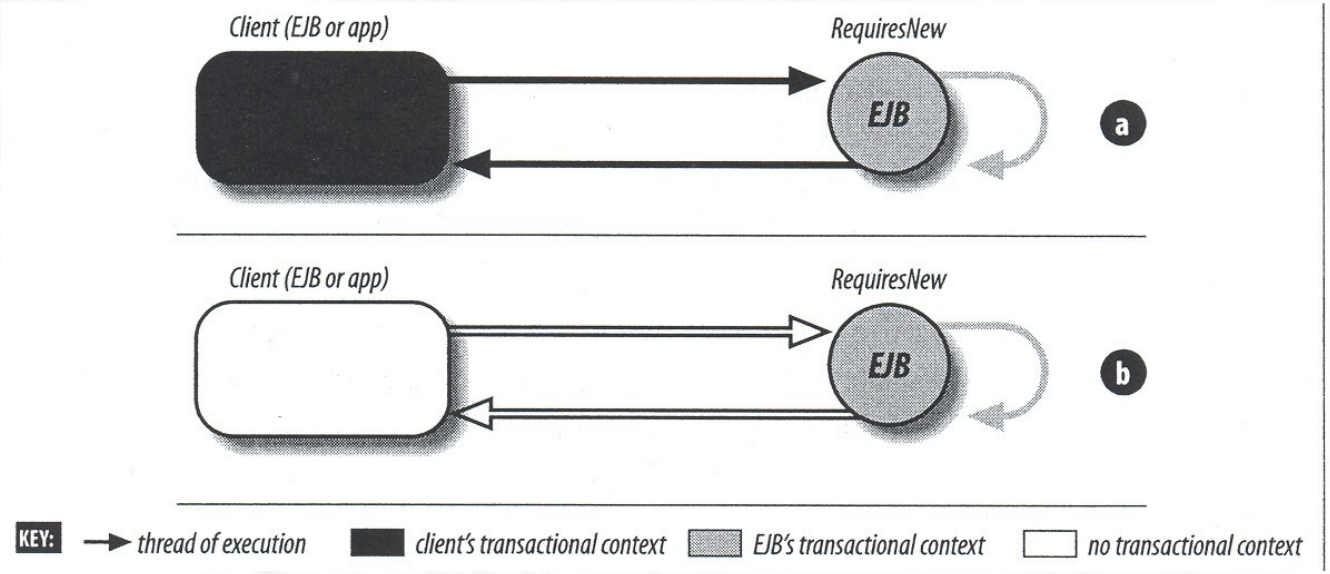
- If client is within transaction, the method executes in the client's transaction
- Client not associated with transaction, container starts a new transaction for this method
- Will work for most cases
- Makes a good default



Transaction Attributes: RequiresNew

- If client running within transaction and calls the method, container does these steps:
  - suspends client's transaction
  - starts new transaction
  - calls method in this new transaction
  - resumes client transaction after method done
- Client not running within transaction, starts one before running method
- Use when method must run in *new* transaction

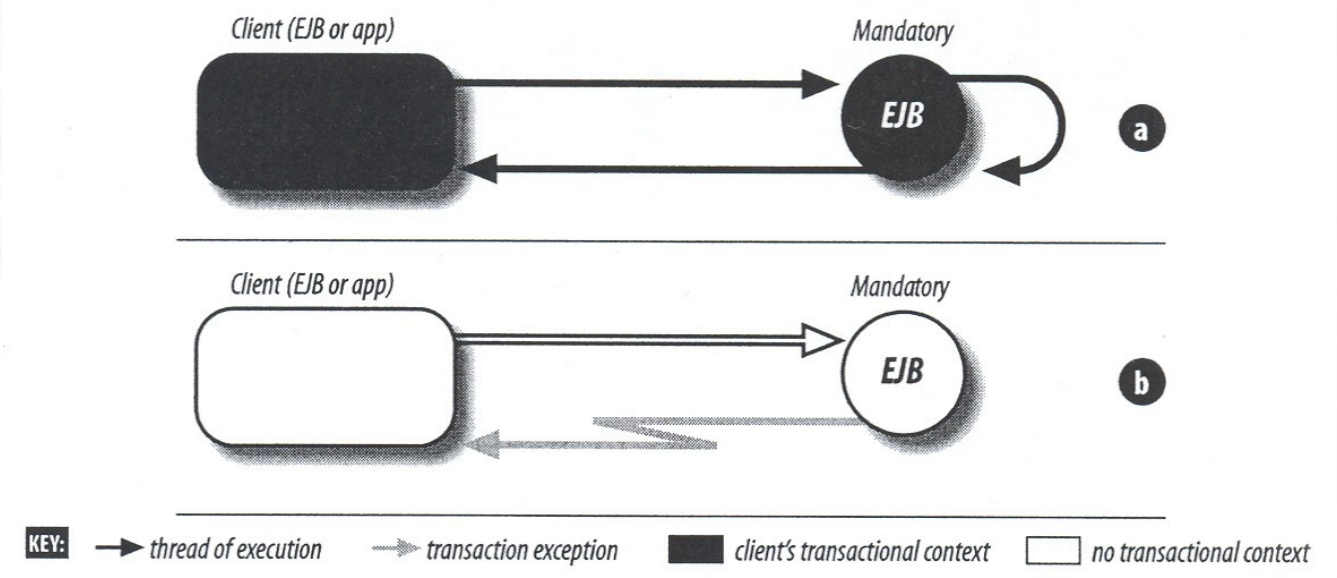
RequiresNew



Transaction Attributes: Mandatory

- If client running within transaction and calls the method
  - method runs in client's transaction
- Client not associated with transaction
  - throws `EJBTransactionRequiredException`
- Use when method must use transaction of the client

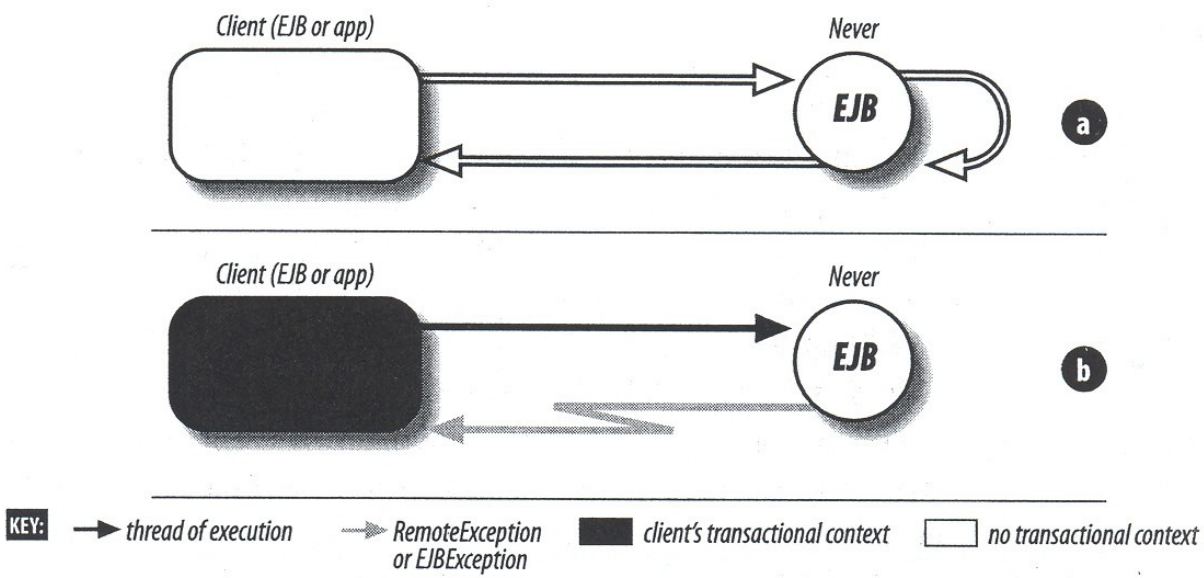
Mandatory


Client (EJB or app)
Mandatory
EJB
a
Client (EJB or app)
Mandatory
EJB
b
KEY:
thread of execution
transaction exception
client's transactional context
no transactional context

Transaction Attributes: Never

- If client running within transaction and calls method
  - throws `EJBException`
- Client not associated with transaction
  - method does not run in transaction

Never


Client (EJB or app)
Never
EJB
a
Client (EJB or app)
Never
EJB
b
KEY:
thread of execution
RemoteException or EJBException
client's transactional context
no transactional context

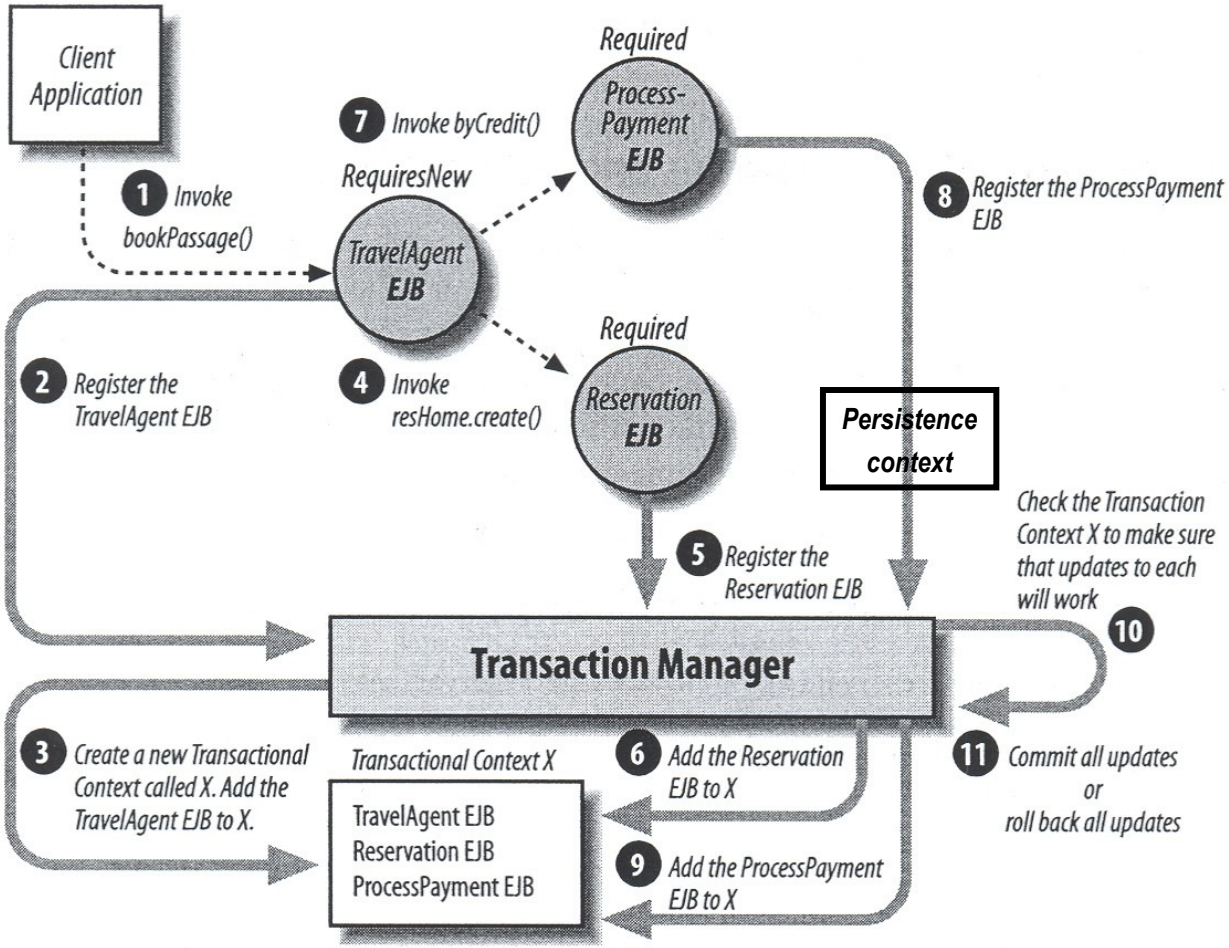
Notes

- CMP (legacy) and Java Persistence beans should only use
  - Required, RequiresNew, and Mandatory
  - ensures database access is in context of a transaction
- Message-driven beans should only use
  - NotSupported or Required
  - others require knowing client transaction and MDB is decoupled from client

Transaction Propagation

- Consider TravelAgent EJB again
  - Reservation and ProcessPayment should be included in same transaction as bookPayment method
    - can do by setting their attributes to required
  - container will handle commit and rollback decisions automatically
  - see later for exception handling

TravelAgent EJB Transactional Context



Transactions and Persistence Context Propagation

- Transaction-scoped entity manager:
  - invoked outside of transaction, creates persistence context for the method
  - invoked inside transaction, persistence context associated with transaction
  - if persistence context already associated with transaction, use that persistence context
  - bean with transation-scoped persistence context calls stateful session bean using extended persistence context: throws EJBException

Transactions and Persistence Context Propagation

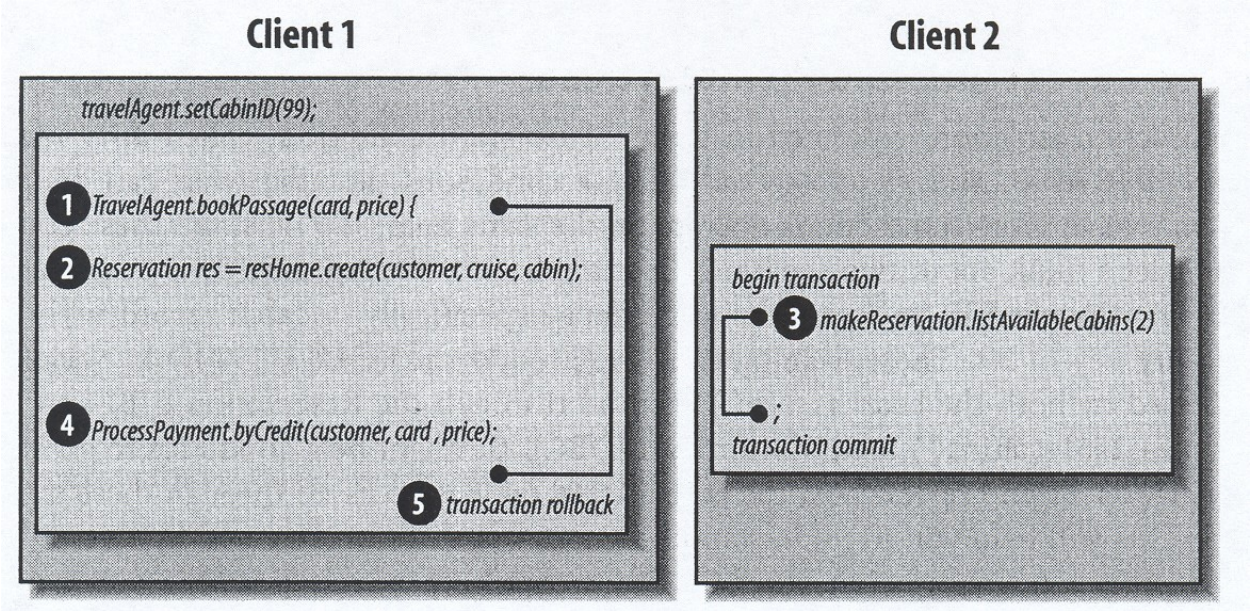
- Stateful session bean with extended persistence context calls other EJB with transaction-scoped persistence context
  - extended context propagated
- If EJB calls EJB which is in a different transaction scope, persistence context not propagated
- Stateful session bean with extended persistence context calls non-injected stateful session bean with extended persistence context
  - EJBException thrown
  - injected stateful session bean shares same persistence context

Isolation and Database Locking

- This is mainly a matter of database theory
- This section considers
  - isolation conditions,
  - database locking and
  - transaction isolation levels
- First look at isolation conditions of dirty, repeatable, and phantom reads
  - what can happen if two or more transactions operate on the same data

Dirty Reads

- Occurs when transaction reads uncommitted changes made by another transaction
  - if first transaction is rolled back, data read by second transaction is invalid

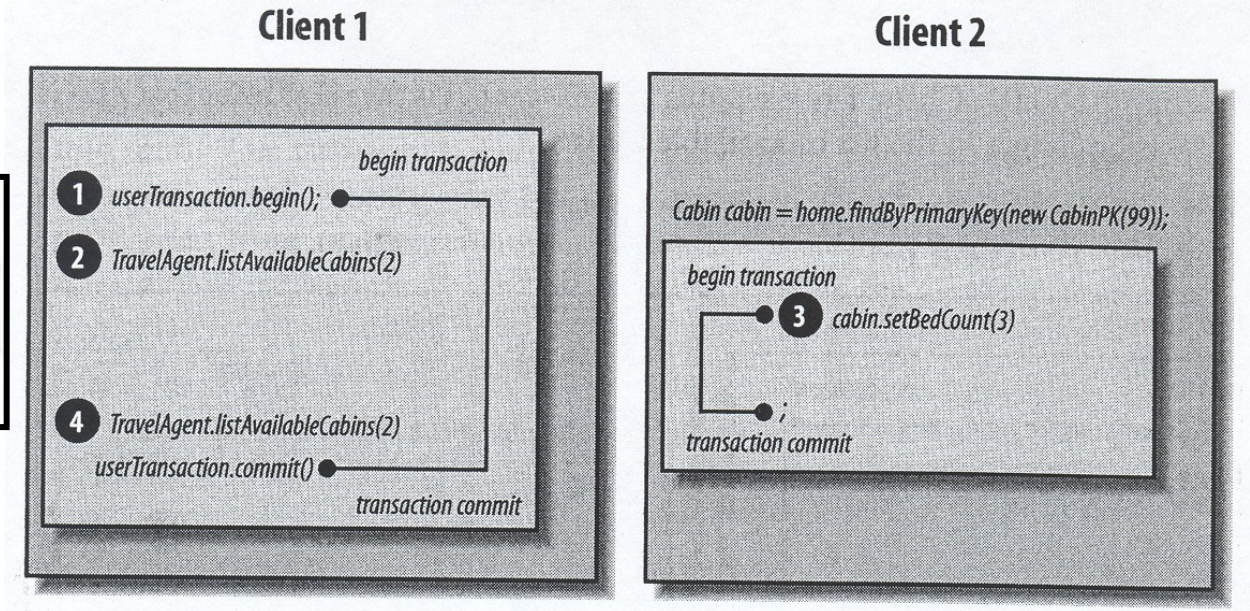


bedcount of 2

Repeatable Reads

- Occurs when data read is guaranteed to look same if read again in same transaction
  - either data is locked, or read in snapshot

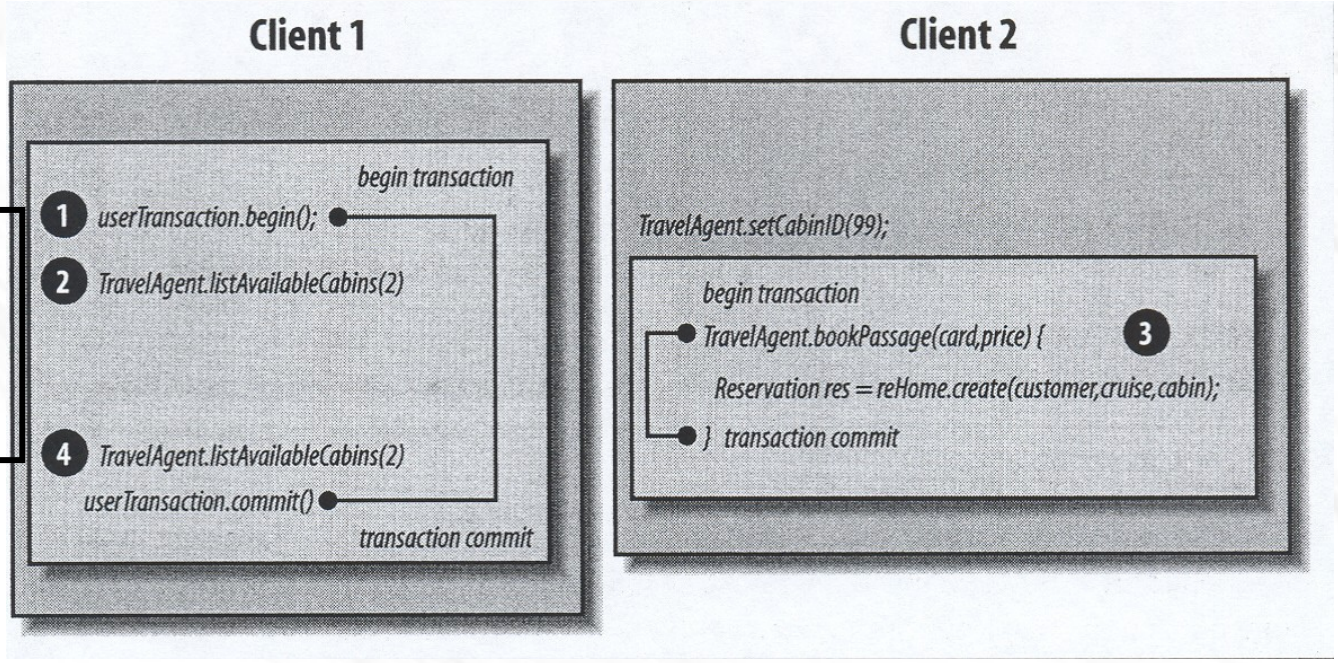
both lists should be same



Phantom Reads

- Occurs when new records are detected by transactions started prior to insert

second list sees new cabin



Database Locks

- Databases normally use locking techniques to enforce isolation levels
  - read locks – prevent other transactions changing data before transaction ends
    - prevents nonrepeatable reads
  - write locks – prevents other transactions from changing data before current one completes
    - allows dirty reads
  - exclusive write locks – prevents other transactions from reading or changing data before current one completes
    - prevents dirty reads

Transaction Isolation Levels

- Read Uncommitted
  - allows dirty reads, nonrepeatable reads, phantom reads
- Read Committed
  - prevents dirty reads
- Repeatable Read
  - prevents dirty and nonrepeatable reads
- Serializable
  - prevents dirty, nonrepeatable, and phantom reads
- EJB Isolation levels set in vendor-specific ways

Balancing Performance Against Consistency

- As isolation levels increase, parallelism decreases
  - so performance suffers
- Higher isolation levels increase data consistency
- Balance performance needs against data consistency to select appropriate levels in each case

Nontransactional Beans

- Beans outside of transaction's scope often provide a stateless service that does not manipulate data in a data store
- Consider leaving these outside of transactions
  - with NotSupported attribute
- Can improve performance

Explicit Transaction Management

- Not covered in lecture due to time constraints
- Read text for an overview
- Not usually recommended in practice

Exceptions and Transactions

- Exceptions have large impact on transaction outcomes
- System exceptions cause transactions to automatically roll back
  - java.lang.RuntimeException and subtypes including EJBException
- Application exceptions do *not* cause transactions to automatically roll back
  - exceptions that do not extend java.lang.RuntimeException or java.rmi.RemoteException
  - exceptions annotated with @ApplicationException

System Exceptions in EJB Method

- Container always
  - rolls back the transaction
  - logs the exception
  - discards the EJB instance
- Impact of discarding stateless session or entity bean instance undetectable for client
- Impact of discarding stateful session instance is more severe
  - lose conversational state and invalidates client reference
- For session beans, an exception is thrown to client
  - EJBException thrown if no transaction
  - EJBTransactionRolledbackException if in a transaction

Application Exceptions

- Thrown for business-logic errors
- Delivered directly to client
- Do not roll back transaction
  - can annotate class or method with @ApplicationException(rollback=true) if application exceptions should rollback transaction
- See (long…) summary table in text

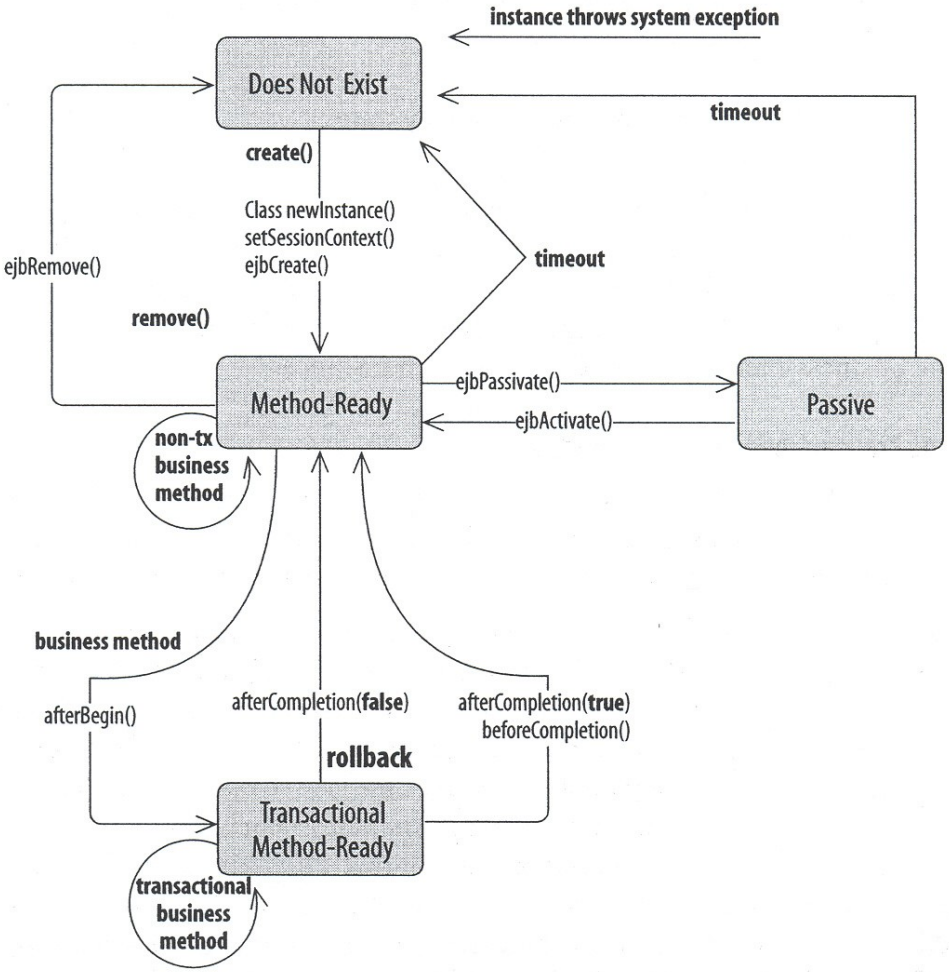
Transactional Session Beans

- Session beans can interact directly with the database
- Stateless session beans have no conversational state
  - so the method boundary is a reasonable transaction boundary
- Stateful session beans *do* have conversational state
  - may want to be aware of transaction boundaries
  - may want to save state only when transaction is committed

javax.ejb.SessionSynchronizatrion Interface

- Allows session bean to receive notification of involvement in transactions
- Implementing methods involves a new state – Transactional Method-Ready
- Interface methods
  - afterBegin(), beforeCompletion(), afterCompletion(boolean committed)

Transactional Stateful Session Beans Lifecycle



Summary

- If Not Sure What to Do With Transactions
  - Specify container-managed transactions
  - Specify **required** transaction attribute for entire bean
- Study the impact of exceptions and use the right ones for the task